

Apunte de Microprogramación

Indice

<u>1. Introducción</u>	2
<u>2. Esquema en bloque de una CPU de un bus</u>	3
2.1 Registros	3
2.2 Bus	4
2.3 Comunicación entre registros y bloques	4
2.4 Compuertas	5
2.5 Unidad de control microprogramada (CU)	6
2.6 Unidad aritmética y lógica (ALU)	6
2.7 Bus de direcciones y de datos	8
2.8 Señales	9
<u>3. Funcionamiento de la CPU</u>	10
3.1 Transferencia de la Información	10
3.2 Ejemplo de una instrucción sin acceso a memoria	11
3.3 Ciclo de Fetch	14
3.4 Contador de microprograma	15
3.5 Instrucciones	16
3.5.1 Sin operandos	16
3.5.2 Con un operando	17
3.5.3 De bifurcación	17
3.5.4 Con dos operandos	18
<u>4. Configuración de dos buses</u>	20
4.1 Registros master-slave (tiempos)	20
4.2 Funcionamiento	21
4.3 Ciclo de Fetch	21
4.4 Instrucciones	22
<u>5. Optimización</u>	23
5.1 Codificadores y decodificadores	23
5.2 Superposición de microoperaciones	24
5.3 Microprogramación horizontal, vertical y mixta	25
5.4 Aplicaciones de una organización mixta	26

1. Introducción

Como sabemos, un sistema se subdivide en distintos niveles de trabajo. Estos son:



Una instrucción de lenguaje ensamblador tiene una relación uno a uno con el nivel de lenguaje máquina y a su vez una instrucción en lenguaje máquina es interpretada por el procesador y ejecutada.

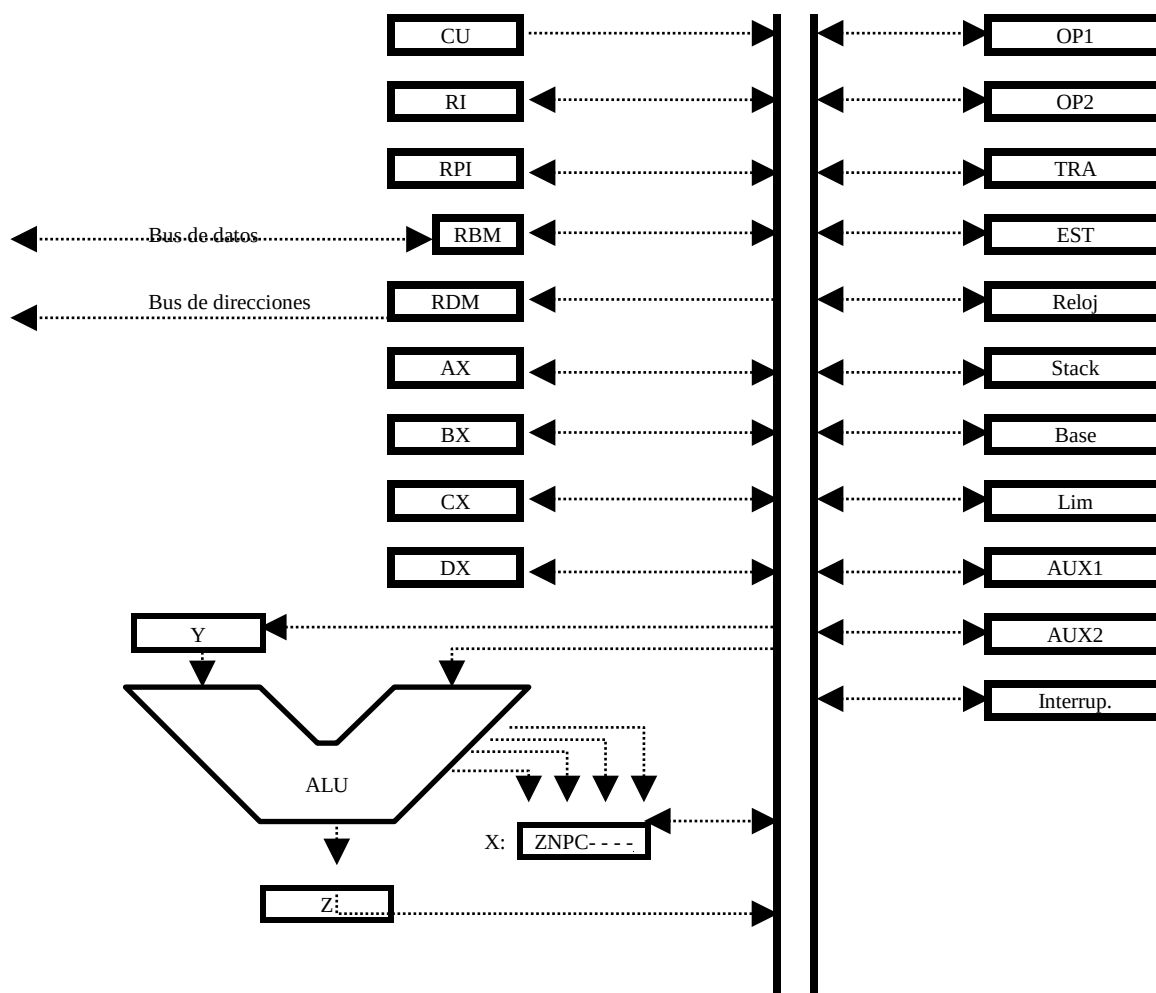
Para ello ira a buscar a la memoria principal una instrucción cargada previamente, la examinará, la ejecutará e irá a buscar la instrucción siguiente, a menos que la ejecución de la instrucción haya modificado la dirección de búsqueda de la proxima instrucción (bifurcación de control).

Para ejecutar una instrucción, el procesador podría descomponer la operación que debe ejecutar en tareas o pasos más pequeños; al ser menor cada tarea es más facil realizarla. Si la Unidad de Control usa esta técnica, a cada una de las tareas en que se descompone una instrucción se la llamará microinstrucción. Un conjunto de microinstrucciones combinadas lógicamente para implementar una instrucción se llama microprograma.

En este capítulo se intentará dar nociones sobre el nivel de microprogramación. Comenzaremos describiendo los dispositivos internos de una CPU convencional y su funcionamiento.

2. Esquema en bloque de una CPU de un bus

En esta sección se definirá la arquitectura y organización de la Unidad de Control microprogramada basada en un bus. Si analizamos el contenido de la figura podemos encontrar los siguientes componentes :



CU : Unidad de control
 RI : Reg. de instrucción
 RPI : Reg. de próxima instrucción
 EST: Reg. de estado
 AUX1, AUX2: Reg. auxiliares

RBM: Reg. buffer de memoria
 RDM: Reg. direc. de memoria
 Y, Z, X: Reg. auxiliares internos
 OP1, OP2: Reg. para operandos
 ALU: Unidad aritmét. y lógica

TRA: Reg. de trabajo
 AX: Reg. de uso general
 BX: Reg. de uso general
 CX: Reg. de uso general
 DX: Reg. de uso general

2.1 Registros

Un registro es un dispositivo en donde se almacena información. Las operaciones que se pueden hacer con registros son dos: Ingresarle una nueva información y Poner a disposición la información almacenada.

La característica más importante del registro es la longitud de palabra, que coincide con la cantidad de bits que es capaz de almacenar simultáneamente. Nosotros, en general, trabajaremos con registros de 16 bits (2 palabras).

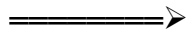
2.2 Bus

Es el medio físico mediante el cual se transmite la información. Un bus es un conjunto de cables donde cada uno de ellos lleva la información de un bit. El valor de cada bit será 0 o 1 de acuerdo a que haya o no tensión.

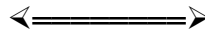
Hay dos formas para transmitir la información: en Serie o en Paralelo. En Serie se transmite un bit a continuación del otro a través de una sola vía de comunicación. Por lo tanto un byte de ocho bits tardará ocho veces el tiempo de transmisión de un bit en completarse. En Paralelo se transmiten los n bits al mismo tiempo mediante n vías de comunicación; como se puede ver, un byte tardará en enviarse el tiempo de transmisión de un bit. Por lo tanto la transmisión en paralelo es más rápida que en serie. En el caso de los buses la transmisión se realiza en paralelo.

Otra característica de los buses es que pueden ser unidireccionales o bidireccionales. Esta condición se representará con la siguiente notación:

Bus unidireccional



Bus bidireccional



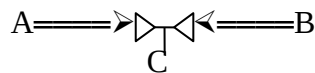
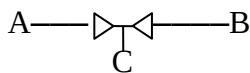
2.3 Comunicación entre registros y bloques

Todos los dispositivos se comunican entre sí por medio de n buses. Si todos los dispositivos estuvieran conectados al bus en forma directa, estarían enviando y recibiendo información al mismo tiempo. Este envío simultáneo de información por el bus produciría la mezcla de las señales eléctricas, provocando la pérdida de la información incluyendo el posible cortocircuito o sobretensión de algún dispositivo.

Para evitar esto es necesario administrar el envío de la información en el bus, y esto se lleva a cabo utilizando compuertas que se conectan entre los dispositivos y el bus. Dichas compuertas son llaves de paso que serán habilitadas en el momento preciso, eliminando la posibilidad de pérdida de la información.

2.4 Compuertas

Existen distintos tipos de compuertas, que permiten o no la circulación de la información. Su funcionamiento es el siguiente: Supongamos que se quiere que la información pase de A a B. Ponemos una señal eléctrica en C, y de acuerdo con su valor, dejará pasar la información de A a B. Es decir que C comanda cuando va a pasar la información y cuando no.



Microprogramación

Los registros son capaces de recibir y enviar información, por ello serán necesarias dos líneas de conexión, una para cada tarea.

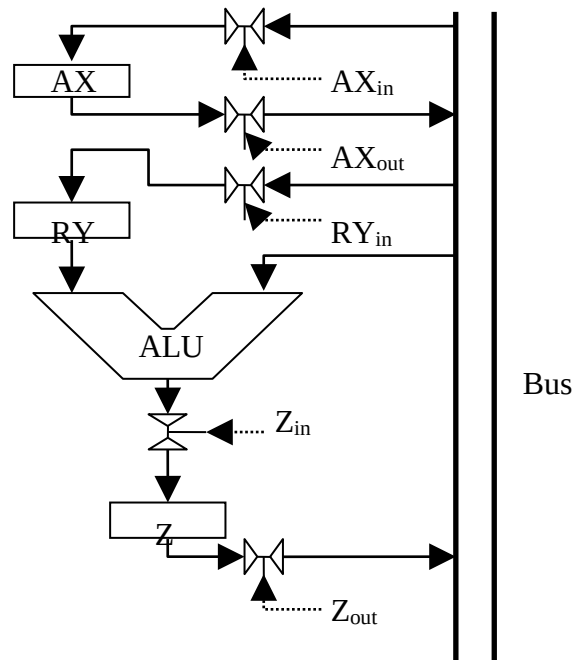
Cada línea de transmisión tendrá una compuerta asociada, la cual a menos que se la invoque, estará siempre desconectada. A los efectos de indicar que una compuerta es invocada, se utilizará la siguiente nomenclatura: para señalar a que dispositivo corresponde se usará la sigla del registro (por ejemplo RPI, AX, RBM), y para establecer el tipo de operación a se efectuará agregaremos un subíndice 'in' para el caso de ingreso de información y 'out' para el envío de la misma.

Por ejemplo:

PRI_{out} : El contenido del registro RPI estará disponible en el bus.

AX_{in} : El contenido del bus se almacenará en el registro AX.

En todos los casos se transmitirá simultáneamente la información de todos los bits, entre los registros y el bus (y viceversa).



2.5 Unidad de control microprogramada (CU)

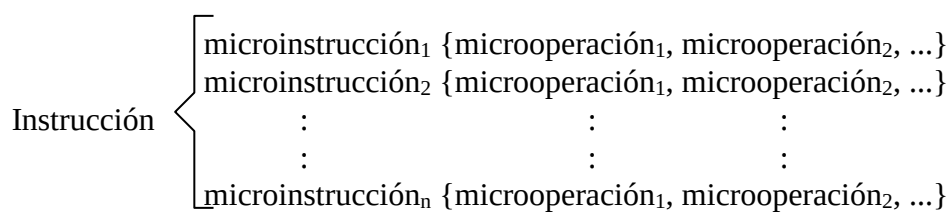
La tarea de la Unidad de Control (Control Unit - CU) es reconocer primero la instrucción a ejecutar y luego ejecutar un microprograma asociado a esa instrucción. Dicho microprograma está conformado por una secuencia de microinstrucciones, cada una de las cuales está compuesta a su vez por microoperaciones. (tales como: habilitar una o varias compuertas, enviar señales de control como ser READ, WRITE, ADDs, AND, etcétera)

Microprogramación

Cada microinstrucción requerirá un tiempo para ejecutarse, en ese tiempo se llevarán a cabo todas las microoperaciones. El tiempo de ejecución de una instrucción será el tiempo de ejecución de todas las microinstrucciones que componen la instrucción.

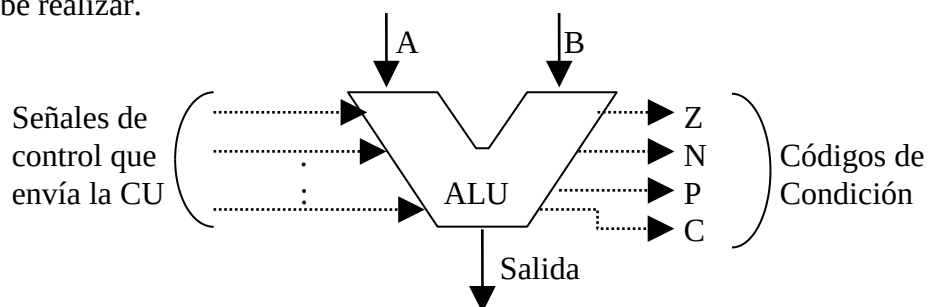
La Unidad de Control hará las siguientes tareas: ir a buscar una instrucción a memoria, decodificar la instrucción y ejecutarla. El proceso de búsqueda de una instrucción (o dato) a memoria es común a todas las instrucciones, es decir a cada microprograma.

Se puede esquematizar cada instrucción como una secuencia de microinstrucciones que a su vez están formadas por microoperaciones:



2.6 Unidad aritmética y lógica (ALU)

La función de la Unidad Aritmética y Lógica (Arithmetic & Logic Unit - ALU) es la de realizar las operaciones aritméticas y lógicas, como ser la SUMA, RESTA, MULTIPLICACIÓN, DIVISIÓN, AND, OR, XOR, etcétera. Para efectuar todas las operaciones, salvo el NOT y el SHIFT, utiliza dos entradas y una salida. Es decir realiza las operaciones sólo con dos operandos a la vez (los de entrada) y devuelve un resultado. La ALU recibe las señales de control que envía la Unidad de Control, sabiendo así la operación que debe realizar.



Además del resultado, la ALU retorna un código de condición de la operación realizada. Dicho código de condición está compuesto por cuatro bits y puede ser :

Z	N	P	C	Significado
1	0	0	0	El resultado de la operación aritmética es cero o el resultado de la operación lógica tiene todos bits en cero.
0	0	1	0	El resultado de la operación aritmética es positivo y sin acarreo o el resultado de la operación lógica tiene todos los bits en 1
0	0	1	1	El resultado de la operación aritmética es positivo y con acarreo
0	1	0	0	El resultado de la operación aritmética es negativo y sin acarreo o el resultado de la operación lógica contiene bits en 0 y en 1

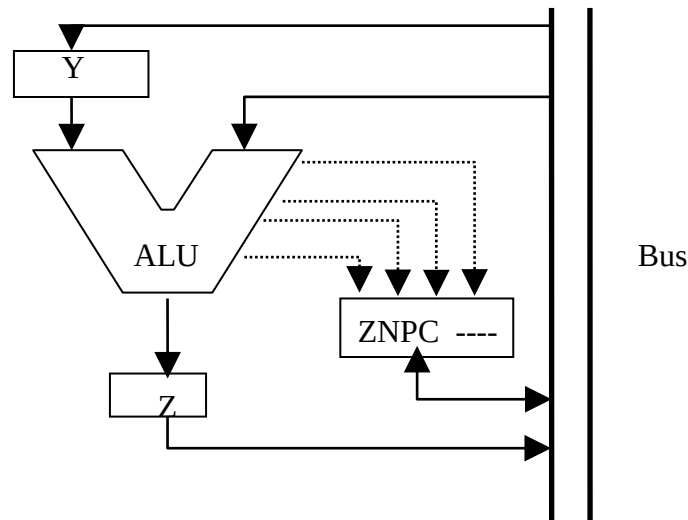
Microprogramación

0	1	0	1	El resultado de la operación aritmética es negativo y con acarreo
---	---	---	---	---

El tiempo que demanda una operación es, a lo sumo, el de una microinstrucción. Debido a que la ALU no puede retener el resultado de la operación efectuada, se la utiliza en combinación con un registro de operación auxiliar (registro Z) al cual se le habilita la compuerta de ingreso en el momento de efectuar la operación.

Para poder trabajar con dos operandos, uno de ellos tendrá que ser previamente almacenado en un registro auxiliar (Registro Y), mientras que el otro estará presente en el bus. Para realizar una operación se habilitará la salida del registro Y y la del otro operando que estará almacenado en otro registro.

Para almacenar los códigos de condición usaremos un registro auxiliar (registro X) de 8 bits, en el cual los primeros 4 bits se destinarán a los códigos Z, N, P, C respectivamente.



Supongamos que queremos realizar una suma, y que los sumandos están almacenados uno en el registro Y y el otro en el registro AX, el resultado de dicha operación quedará en el registro Z. Para ello la microinstrucción a utilizar sería:

$$Z = RY + AX \quad \equiv \quad \text{ADDs, } AX_{\text{out}}, Z_{\text{in}}$$

Nota: las señales de control como ADDs que utilizan dos operandos habilitan la compuerta de salida de RY, es decir, dan simultáneamente las órdenes a la ALU y a RY_{out} .

Vemos que la anterior es una microinstrucción compuesta por tres microoperaciones que se realizan simultáneamente que son:

ADDs: Señal de control a la ALU y habilitación de la salida de información a Y_{out} .

Microprogramación

AX_{out} : Habilitación de la salida de información del registro AX.

Z_{in} : Habilitación de la entrada de información al registro Z.

A continuación indicaremos las operaciones que puede realizar la ALU :

Operación	Microoperación	Resultado
Suma	ADDs, ADDc, ADDsa, ADDca	Bus + [Y]
Resta	SUBs, SUBc, SUBsa, SUNca	Bus - [Y]
Multiplicación	MULTs, MULTc	Bus * [Y]
División	DIVs, DIVc	Bus / [Y]
Decalaje	SHIFT _l , SHIFT _r	Decala Bus
AND Lógico a bit	AND	Bus ^ [Y]
OR Lógico a bit	OR	Bus v [Y]
XOR Lógico a bit	XOR	Bus <u>v</u> [Y]
NOT Lógico a bit	NOT	~ Bus

donde los subíndices indican :

s	Operación aritmética sin signo y sin considerar o modificar el acarreo
c	Operación aritmética en notación complemento y sin considerar o modificar el acarreo
sa	Operación aritmética sin signo y considerando y modificando el acarreo
ca	Operación aritmética en notación complemento y considerando y modificando el acarreo

En una operación de suma con acarreo, si el bit de carry está en uno, se incrementa el resultado y en una de resta, con las mismas condiciones, se decrementa.

2.7 Bus de direcciones y de datos

Hay dos registros que contendrán la información para comunicarse con el medio externo, estos son:

RBM : Registro base de memoria

RDM : Registro de dirección de memoria

El primero contendrá la información que fue recibida o que va a ser transmitida y el segundo contendrá la dirección desde donde la información será recibida o a donde será enviada. El RBM estará conectado al bus de datos y el RDM al de direcciones.

Para efectuar una lectura de memoria deberá primero cargar la dirección de memoria en el RDM y luego enviar una señal de pedido de lectura a memoria y esperar a que se realice la

operación. Una vez que se recibe la señal de que la operación se ha completado el contenido de la lectura solicitada estará almacenado en el RBM.

Como el acceso a memoria se hará de a un byte (una palabra), dicho bus contendrá 8 “cables” y el RBM almacenará sólo 8 bits. Debido a que estamos trabajando con un mapa de memoria de 0000h a FFFFh necesitamos 2 bytes para poder representar todas las direcciones, por lo cual el bus de direcciones será de 16 bits y el registro RDM también.

2.8 Señales

Las señales brindan información acerca del control de lo que se debe hacer. Como vimos antes, la Unidad de Control envía señales a la ALU sobre qué operación debe efectuar. Además envía señales a la memoria sobre si debe hacer una operación de lectura o escritura. Hay otras señales externas como ser la que indica la finalización de la operación con la memoria.

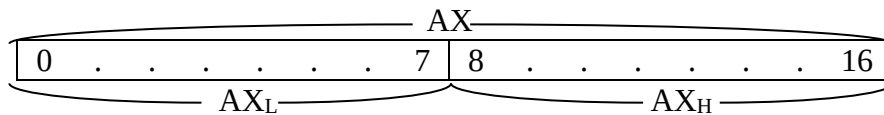
3. Funcionamiento de la CPU

En esta sección analizaremos el funcionamiento de la CPU en cuanto a la ejecución de instrucciones.

3.1 Transferencia de la Información

La transferencia se puede hacer de dos formas: en una se transfiere el contenido de un registro entero (16 bits) mientras que en la otra se puede transmitir medio registro (8 bits); en este caso se podrá trabajar con los primeros 8 bits o con los últimos 8.

En el primer caso nos referiremos al registro como fue expresado anteriormente y en el segundo se agregarán los subíndices 'L' y 'H' según si se quiere operar con los primeros 8 bits o con los últimos 8 ('L' = Low, 'H' = High).



Veamos cómo se puede hacer para copiar información de un registro a otro, por ejemplo del AX al BX. Se habilita la compuerta de salida de AX, haciéndose de esta manera la información presente en el bus, y se abre la compuerta de entrada de BX, permitiendo de esta manera que la información del bus quede almacenada.

Todo esto debe ocurrir al mismo tiempo debido a que en caso contrario la información de AX no se mantendrá en el bus y BX tomará cualquier valor. El tiempo en el cual se efectuarán estas dos microoperaciones es el de una microinstrucción. En este caso la microinstrucción a realizar es :

AX_{out}, BX_{in}

Como vemos, todas las microoperaciones que componen una microinstrucción se escriben en una misma línea sin importar el orden dentro de la microinstrucción, por lo cual es equivalente escribir : AX_{out}, BX_{in} a BX_{in}, AX_{out} .

Ahora analicemos el caso de copiar primero de AX a BX y luego de AX a CX :

AX_{out}, BX_{in}
 AX_{out}, CX_{in}

Como vemos para realizar esta tarea se necesitan dos microinstrucciones. Otras formas de obtener el mismo resultado son :

AX_{out}, BX_{in}	Equivalente	AX_{out}, CX_{in}	Equivalente	AX_{out}, CX_{in}
BX_{out}, CX_{in}	a	AX_{out}, BX_{in}	a	CX_{out}, BX_{in}

Supongamos que se quisieran ahorrar microoperaciones, se podría haber planteado :

$$\begin{array}{l} AX_{out}, BX_{in} \\ CX_{in} \end{array}$$

¿Que pasaría en este caso? BX tendría el contenido de AX, pero CX tendría un valor indefinido. ¿Por que? La respuesta es que el contenido de AX se mantendrá en el bus únicamente por el tiempo que tarde la primer microinstrucción, una vez que esta haya concluido, nadie puede asegurar cual es el contenido del bus, por lo tanto el contenido final de CX.

Veamos ahora como es posible reducir efectivamente la cantidad de microinstrucciones a realizar (y a la vez de microoperaciones). Aprovechando el momento en que la información de AX está disponible en el bus, nada nos impide almacenar esta información en BX y en CX a la vez, de la siguiente manera :

$$AX_{out}, BX_{in}, CX_{in}$$

Como se puede observar, la diferencia principal entre la primera forma de copiar AX a BX y CX y esta última es que en esta se ahorra el tiempo de una microinstrucción.

Ejercicios :

¿Cual es el contenido del registro AX despues de ejecutar cada una de las siguientes microinstrucciones?

- | | | | |
|------|---|-----|--|
| i) | AX_{out}, BX_{in}
AX_{in}, CX_{out}
BX_{out}, CX_{in} | ii) | AX_{out}, BX_{in}
BX_{out}, AX_{in} |
| iii) | AX_{out}, BX_{in}
AX_{in}, BX_{in} | iv) | $BX_{out}, AX_{in}, CX_{out}$
BX_{out}, CX_{in} |

Nota : No hay errores de tipeado ... ¿Otros? Tal vez.

3.2 Ejemplo de una instrucción sin acceso a memoria

Aqui nos referiremos sólo a una instrucción que en su ejecución no hará acceso a memoria. La instrucción es copiar el contenido de un registro a otro.

MOV AX, BX es equivalente a $[AX] \leftarrow [BX]$

Microprogramación

Primero se debe buscar la instrucción a memoria, luego decodificarla y por último ejecutarla. A nivel de microprograma será:

1. RPI_{out} , RDM_{in}
2. READ
3. $WAIT_{fmc}$
4. CLEAR Y
5. RPI_{out} , $CARRY_{in=1}$, $ADDsa$, Z_{in}
6. Z_{out} , RPI_{in}
7. RBM_{out} , RI_{Lin}
8. RPI_{out} , RDM_{in}
9. READ
10. $WAIT_{fmc}$
11. CLEAR Y
12. RPI_{out} , $CARRY_{in=1}$, $ADDsa$, Z_{in}
13. Z_{out} , RPI_{in}
14. RBM_{out} , RI_{Hin}
- ===== Decodificación =====
15. BX_{out} , AX_{in}
16. END

1. RPI_{out} , RDM_{in}

Se pasa la dirección de búsqueda de la próxima instrucción que está en el RPI al registro de dirección de memoria, el cual está conectado a la memoria por medio del bus de direcciones. Es una operación de lectura/escritura la memoria va a buscar la dirección de trabajo al bus de direcciones.

2. READ

Efectúa la lectura de la memoria. La operación READ lo que hace es enviar una señal de lectura a la memoria, abrir la compuerta de salida que está entre el RDM y el bus de direcciones permitiendo así que el contenido del RDM esté en dicho bus, y abrir la compuerta de entrada que está entre el bus de datos y el RBM permitiendo así que el contenido del bus de datos se almacene en el RBM.

La memoria al recibir la señal efectuará la operación de lectura tomando como dirección de búsqueda la del bus de direcciones y hará presente el contenido de dicha dirección en el bus de datos.

3. $WAIT_{fmc}$

Hasta que la operación con la memoria no se haya terminado se mantendrán las compuertas abiertas de salida del RDM y de entrada del RBM. El WAIT bloquea la CPU hasta recibir la señal que envía la memoria y entonces cierra las compuertas abiertas anteriormente. (fmc = función de memoria completada).

4. CLEAR Y

De esta manera se ponen todos los bits del registro Y en cero.

5. RPI_{out} , $CARRY_{in}=1$, $ADDsa$, Z_{in}

El bit asociado al carry se pone en uno, esto implica que en una operación de suma o resta con acarreo modificará en uno el resultado. El $ADDsa$ es una señal de control que le indica a la ALU la operación que debe realizar. La ALU tomará el contenido del registro Y y del bus, por eso simultáneamente se hace RPI_{out} . Por lo tanto va a sumar :

$$\begin{array}{r} \text{Registro Y} \quad \quad 0 \\ \text{Bus [RPI]} \quad \quad RPI \\ \hline \text{Carry} \quad \quad 1 \\ \hline \text{Registro Z} \quad \quad [RPI]+1 \end{array}$$

El resultado de la operación se almacenará en el registro Z, si la compuerta del entrada del mismo estaba habilitada. Notar que si no se abre esta compuerta el resultado de la operación se pierde.

6. Z_{out} , RPI_{in}

Se pasan el resultado de la operación anterior al RPI.

7. RBM_{out} , RI_{Lin}

El contenido del RBM se carga en la parte baja del registro de instrucción (RI_L), es decir en el primer byte.

Lo que hacen los pasos 4, 5 y 6 es incrementar el RPI en uno. El resto de los pasos efectúan la operación de búsqueda de la instrucción en memoria para luego cargarla en el registro de instrucción. Con lo que hicimos hasta ahora se cargó el primer byte del RI, por lo que falta cargar el segundo byte. Para esto se repiten los siete pasos anteriores con la diferencia que en el último de ellos se carga la parte alta del RI.

8. RPI_{out} , RDM_{in}

9. READ

10. $WAIT_{fmc}$

11. CLEAR Y

12. RPI_{out} , $CARRY_{in}=1$, $ADDsa$, Z_{in}

13. Z_{out} , RPI_{in}

14. RBM_{out} , RI_{Hin}

===== Decodificación =====

En este punto se terminó la operación de búsqueda de la instrucción a memoria y se realiza la decodificación o interpretación del código de operación que dará el punto de comienzo del microprograma que ejecutará la instrucción.

15. BX_{out} , AX_{in}

Se copia el contenido de BX a AX.

16. END

Esta microoperación avisa que terminó de ejecutarse la instrucción, y la Unidad de Control arranca de nuevo para buscar la próxima instrucción.

Nota: En este ejemplo no se tuvieron en cuenta los códigos de condición para no hacer compleja la comprensión del microprograma.

3.3 Ciclo de Fetch

Las primeras 14 instrucciones del microprograma anterior conforman en Ciclo de Fetch de instrucciones. Como estas microinstrucciones se ejecutan por cada instrucción que la CPU ejecutará, es importante tratar de optimizar este ciclo cuanto sea posible.

Analizando nuevamente el ejemplo anterior vemos que se puede mejorar el ciclo de búsqueda (disminuir la cantidad de microinstrucciones). Las microinstrucciones 2 y 3 se pueden hacer simultáneamente ya que se puede hacer el pedido de lectura y esperar a que se complete. De la misma manera se puede poner el Registro Y en cero y en la misma microinstrucción realizar la operación de suma. Estos pasos se repiten en las microinstrucciones 9 y 10 y 11 y 12. Con estos cambios el ciclo de FETCH sería :

1. RPI_{out}, RDM_{in}
 2. READ, $WAIT_{fmc}$
 3. CLEAR Y, $RPI_{out}, CARRY_{in=1}, ADDsa, Z_{in}$
 4. Z_{out}, RPI_{in}
 5. RBM_{out}, RI_{Lin}
 6. RPI_{out}, RDM_{in}
 7. READ, $WAIT_{fmc}$
 8. CLEAR Y, $RPI_{out}, CARRY_{in=1}, ADDsa, Z_{in}$
 9. Z_{out}, RPI_{in}
 10. RBM_{out}, RI_{Hin}
- ===== Decodificación =====

Las microinstrucciones 2 y 3 se pueden realizar al mismo tiempo debido a que no hay conflicto entre la señal de lectura, la suma y el uso del bus, quedando el ciclo de búsqueda así :

1. RPI_{out}, RDM_{in}
 2. READ, $WAIT_{fmc}, CLEAR Y, RPI_{out}, CARRY_{in=1}, ADDsa, Z_{in}$
 3. Z_{out}, RPI_{in}
 4. RBM_{out}, RI_{Lin}
 5. RPI_{out}, RDM_{in}
 6. READ, $WAIT_{fmc}, CLEAR Y, RPI_{out}, CARRY_{in=1}, ADDsa, Z_{in}$
 7. Z_{out}, RPI_{in}
 8. RBM_{out}, RI_{Hin}
- ===== Decodificación =====

Microprogramación

Tratemos ahora de unir las microinstrucciones 2 y 3. Vemos que no hay un conflicto de señales, pero si lo hay en el uso del bus ya que uno pretendía que en el bus estuviera el contenido del RPI y el contenido de Z, lo que carece de sentido. Un caso análogo sería tratar de juntar la microinstrucción 3 con la 4 o la 2 con la 4.

Intentaremos unir las microinstrucciones 1 y 2. En ambas el contenido del bus es el mismo, por lo tanto no hay incompatibilidad.

1. RPI_{out} , RDM_{in} , READ, $WAIT_{fmc}$, CLEAR Y, $CARRY_{in=1}$, $ADDsa$, Z_{in}
 2. Z_{out} , RPI_{in}
 3. RBM_{out} , RI_{Lin}
 4. RPI_{out} , RDM_{in} , READ, $WAIT_{fmc}$, CLEAR Y, $CARRY_{in=1}$, $ADDsa$, Z_{in}
 5. Z_{out} , RPI_{in}
 6. RBM_{out} , RI_{Hin}
- ===== Decodificación =====

En este punto notamos que no se pueden superponer más microinstrucciones debido a que cada una utiliza una información distinta en el bus. Este microprograma demoraría 6 tiempos (de ejecución de una microinstrucción) en llevarse a cabo, pero como tiene dos operaciones con la memoria, la microoperación WAIT retardará su ejecución el tiempo que tarde la memoria en responder.

Podríamos hacer que ese retraso afecte lo menos posible y esto lo conseguiremos reubicando la microoperación $WAIT_{fmc}$ en la microoperación anterior a que uno necesite la información pedida. Realizando esta última modificación el Ciclo de Fetch quedará :

1. RPI_{out} , RDM_{in} , READ, CLEAR Y, $CARRY_{in=1}$, $ADDsa$, Z_{in}
 2. Z_{out} , RPI_{in} , $WAIT_{fmc}$
 3. RBM_{out} , RI_{Lin}
 4. RPI_{out} , RDM_{in} , READ, CLEAR Y, $CARRY_{in=1}$, $ADDsa$, Z_{in}
 5. Z_{out} , RPI_{in} , $WAIT_{fmc}$
 6. RBM_{out} , RI_{Hin}
- ===== Decodificación =====

Con esto hemos alcanzado una versión muy optimizada del Ciclo de Fetch, hecho fundamental dado que el mismo se ejecutará cada vez que la CPU ejecute cualquier instrucción.

Es importante tratar de optimizar todas las instrucciones que se ejecutan (disminuyendo la cantidad de microinstrucciones necesarias) juntando microinstrucciones y/o reubicandolas, pero se debe tener el mayor de los cuidados para no cambiar el sentido de la lógica de la instrucción original.

3.4 Contador de microprograma

Microprogramación

Para llevar el control sobre la microinstrucción del microprograma que se debe ejecutar, hay un contador (mPC : MicroProgram counter) dentro de la Unidad de Control que dirá el número de microinstrucción que se debe efectuar. Del análisis del Fetch se desprende que siempre comienza con un valor inicial que se irá incrementando de a uno con cada microinstrucción que se ejecute.

¿Que pasa una vez que se ejecutó la microinstrucción 6? Debe ejecutar la primer microinstrucción del microprograma correspondiente a la instrucción que levantó. Dicha microinstrucción tendrá un número dentro de la Unidad de Control, el cual estará relacionado con el código de operación de la instrucción.

Ello significa que despues de ejecutar la microinstrucción 6, se podrá ejecutar la microinstrucción 213, o la 65, o la 187, ... de acuerdo a la instrucción levantada.

Nota: Debido a las características de nuestro assembler y para no hacer complejo el mapa de microinstrucciones de la Unidad de Control adoptaremos la notación que el comienzo de cada microprograma coincida con la microinstrucción siguiente al Fetch. Para poder efectuar bifurcaciones incondicionales o saltos tenemos la microinstrucción `BRANCH n` donde 'n' es un número que indica la microinstrucción a ejecutar.

Despues de la ejecución de un microprograma debe empezar el Fetch de la siguiente instrucción, para lo cual habrá que poner el mPC en cero y esto lo hacemos con la microoperación `END`.

3.5 Instrucciones

Se mostrará aqui el microprograma de algunos tipos de instrucciones como ejemplo.

3.5.1 Instrucciones sin operandos en memoria

Las instrucciones que no usan operandos en memoria son aquellas que trabajan sólo con registros. La mayoría de las intrucciones modifican los códigos de condición.

Luego de cada operación de la ALU se modifican los códigos de condición del registro `AX`; cuando interese retener su valor debe ser transferirlo al registro `EST`. Como no se puede transferir medio byte (4 bits) primero se cargará el registro `AX`, con la parte alta del registro `EST`; luego se opera con la ALU, y por último se copia el contenido de `AX` en la parte alta de `EST`.

ADD CX, BX (Suma de registros)

ADD	CX	BX
-----	----	----

FETCH

===== Decodificación =====

- 6) EST_{Hout}, AX_{in}
- 7) CX_{out}, Y_{in}
- 8) $BX_{out}, ADDsa, Z_{in}$
- 9) AX_{out}, EST_{Hin}
- 10) Z_{out}, CX_{in}, END

3.5.2 Instrucciones con un operando en memoria

En la ejecución de este tipo de instrucciones se debe leer un operando de la memoria, para lo cual se harán dos lecturas (pasos 6 a 11) que se almacenarán en el registro OP1. Una vez obtenido el operando, se lo debe resolver, para lo cual se utilizará una microrutina, que trabajará con el registro auxiliar TRA (del que tomará el operando y en el cual dejará la dirección obtenida).

Para llamar a la microrutina se deberá hacer una bifurcación a su microinstrucción de comienzo (si suponemos que la misma empieza en la dirección 300, será BRANCH 300), y para retornar de la misma se debe almacenar el número de la próxima microinstrucción a ejecutar en el contador de microprograma auxiliar (por ejemplo: $PC_{aux}=60$). Para volver, la microrutina bifurca a PC_{aux} (BRANCH PC_{aux}).

MOV ROT, DX (Carga de registro con memoria)

MOV	DX	Dir. ROT
-----	----	----------

FETCH

===== Decodificación =====

- 6) $RPI_{out}, RDM_{in}, READ, CLEAR Y, CARRY_{in=1}, ADDsa, Z_{in}$
- 7) $Z_{out}, RPI_{in}, WAIT_{fmc}$
- 8) $RBM_{out}, OP1_{Lin}$
- 9) $RPI_{out}, RDM_{in}, READ, CLEAR Y, CARRY_{in=1}, ADDsa, Z_{in}$
- 10) $Z_{out}, RPI_{in}, WAIT_{fmc}$
- 11) $RBM_{out}, OP1_{Hin}$
- 12) $OP1_{out}, TRA_{in}, PC_{aux}=50$
- 13) BRANCH 300
- 50) $TRA_{out}, RDM_{in}, READ, WAIT_{fmc}$
- 51) RBM_{out}, DX_{Lin}
- 52) $TRA_{out}, CLEAR Y, CARRY_{in=1}, ADDsa, Z_{in}$
- 53) $Z_{out}, RDM_{in}, READ, WAIT_{fmc}$
- 54) RBM_{out}, DX_{Hin}, END

Ejercicio : En la instrucción anterior no se tuvieron en cuenta los códigos de condición. Una vez concluida la lectura del presente capítulo, implementar las microinstrucciones correspondientes.

3.5.3 Instrucciones de bifurcación

Lo que hacen estas instrucciones es modificar el RPI con el contenido de un operando.

JMP ROT (Bifurcación incondicional)

JMP	Dir. ROT
-----	----------

FETCH

===== Decodificación =====

6) RPI_{out}, RDM_{in}, READ, CLEAR Y, CARRY_{in=1}, ADDsa, Z_{in}

7) Z_{out}, RPI_{in}, WAIT_{fmc}

8) RBM_{out}, OP1_{Lin}

9) RPI_{out}, RDM_{in}, READ, CLEAR Y, CARRY_{in=1}, ADDsa, Z_{in}

10) Z_{out}, RPI_{in}, WAIT_{fmc}

11) RBM_{out}, OP1_{Hin}

12) OP1_{out}, TRA_{in}, PC_{aux=80}, BRANCH 300

80) TRA_{out}, RPI_{in}, END

3.5.4 Instrucciones con dos operandos en memoria

En la siguiente instrucción, los pasos 6 a 11 y 12 a 17 respectivamente almacenan los operandos ROT1 en OP1 y ROT2 en OP2. Los pasos 43 a 54 forman un ciclo que decrementa la cantidad de bytes a mover, leyendo y grabando el byte especificado en las direcciones OP1 y OP2. Debido a que el código de condición resultante de la ejecución de la instrucción depende del contenido de cada byte, será necesario mantener el código de cada byte.

En la microinstrucción 53 se llamará a una microrutina que está en la microinstrucción 200, la cual efectúa la tarea de almacenar el código de condición de cada byte. Esta rutina toma como parámetro el registro AUX1 y operará con el registro AUX2 en donde dejará almacenado el código de condición acumulado.

En la microinstrucción 80 se invoca a otra microrutina la cual está en la microinstrucción 250 y cuya tarea es la de tomar el contenido del registro AUX2 y determinar su código de condición, almacenándolo en la parte alta de EST.

MOVS ROT1, ROT2 (Mover strings)

MOVS	ROT1	ROT2
------	------	------

FETCH

===== Decodificación =====

6) RPI_{out}, RDM_{in}, READ, CLEAR Y, CARRY_{in=1}, ADDsa, Z_{in}

7) Z_{out}, RPI_{in}, WAIT_{fmc}

8) RBM_{out}, OP1_{Lin}

9) RPI_{out}, RDM_{in}, READ, CLEAR Y, CARRY_{in=1}, ADDsa, Z_{in}

10) Z_{out}, RPI_{in}, WAIT_{fmc}

11) RBM_{out}, OP1_{Hin}

12) RPI_{out}, RDM_{in}, READ, CLEAR Y, CARRY_{in=1}, ADDsa, Z_{in}

13) Z_{out}, RPI_{in}, WAIT_{fmc}

Microprogramación

- 14) RBM_{out}, OP2_{Lin}
- 15) RPI_{out}, RDM_{in}, READ, CLEAR Y, CARRY_{in=1}, ADD_{sa}, Z_{in}
- 16) Z_{out}, RPI_{in}, WAIT_{fmc}
- 17) RBM_{out}, OP2_{Hin}
- 18) OP1_{out}, TRA_{in}, PC_{aux=30}, BRANCH 300
- 30) TRA_{out}, OP1_{in}
- 31) OP2_{out}, TRA_{in}, PC_{aux=40}, BRANCH 300
- 40) TRA_{out}, OP2_{in}
- 41) CLEAR TRA, CLEAR AUX1, CLEAR AUX2
- 42) RI_{Hout}, TRA_{Hin}
- 43) CLEAR Y, TRA_{out}, CARRY_{in=1}, SUB_{ca}, Z_{in}
- 44) IF N, BRANCH 80
- 45) Z_{out}, TRA_{in}
- 46) OP2_{out}, RDM_{in}, READ, CLEAR Y, CARRY_{in=1}, ADD_{sa}, Z_{in}
- 47) Z_{out}, OP2_{in}, WAIT_{fmc}
- 48) OP1_{out}, RDM_{in}, READ, CLEAR Y, CARRY_{in=1}, ADD_{sa}, Z_{in}
- 49) Z_{out}, OP1_{in}, WAIT_{fmc}
- 50) RBM_{out}, Y_{in}
- 51) CLEAR RBM
- 52) RBM_{out}, SUB_s
- 53) X_{out}, AUX1_{Lin}, PC_{aux=54}, BRANCH 200
- 54) BRANCH 43
- 80) PC_{aux=81}, BRANCH 250
- 81) END

La microoperación IF (línea 44) funciona de la siguiente manera. Consulta el bit de los códigos de condición del registro AX, y en el caso de coincidir hace una bifurcación a donde le indique el BRANCH. En caso contrario, continuará con la siguiente microinstrucción. El IF usa los códigos de condición Z, P, ~P, N, ~N, C y ~C. Esta microoperación no se puede ejecutar conjuntamente con otras.

4. Configuración de dos buses

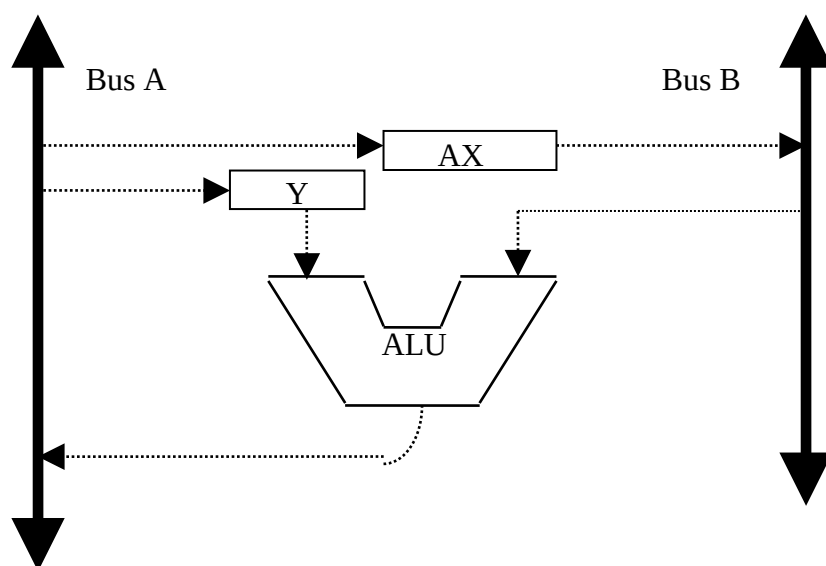
4.1 Registros master-slave (tiempos)

Estos registros tienen la propiedad de que en el tiempo de una microinstrucción pueden poner presente su contenido en un bus y a la vez almacenar la información que hay en el otro bus, es decir, en una microinstrucción pueden efectuarse las microoperaciones : RX_{in} , RX_{out} .

Para ello se descompone el tiempo de cada microinstrucción en tres partes :

- Se habilitan las compuertas de salida (out).
- Retardo de la operación de la ALU.
- Se habilitan las compuertas de entrada (in).

Al culminar el tiempo de la microinstrucción todas las compuertas son deshabilitadas, por lo tanto despues del tiempo 3 se cerrarán las compuertas abiertas en 1 y 3. Supongamos que queremos sumarle 1 a un registro en la siguiente configuración :



La microoperación sería la siguiente :

AX_{out} , $CARRY_{in}=1$, $CLEAR Y$, $ADDsa$, AX_{in} , ALU_{out}

Analizando los contenidos de los buses A y B en los 3 tiempos antes descriptos, estos son:

	Bus A	Bus B
Tiempo 1	?	AX
Tiempo 2	$AX + 1$	AX
Tiempo 3	$AX + 1$	$AX + 1$

4.2 Funcionamiento

El Bus A contendrá la información que se almacenará en los registros. En el Bus B estará presente la información de algunos de los registros. Para que la información pase del Bus B al Bus A se utiliza una compuerta G que será habilitada cuando sea necesario. En forma análoga se utiliza una compuerta ALU_{out} para transferir de la ALU al Bus A.

A continuación se harán algunas operaciones para esta configuración :

Copiar el contenido del registro AX a BX : $AX_{out}, G_{on}, BX_{in}$

Aquí la información de AX se coloca en el bus B, se habilita la compuerta de comunicación entre los buses y la información pasa a estar presente en el Bus A del cual toma la información BX.

Sumar los registros CX y DX y almacenar el resultado en AX

CX_{out}, G_{on}, Y_{in}

$DX_{out}, ADDsa, ALU_{out}, AX_{in}$

Observamos que la ALU recibe el contenido del Bus B y al resultado de la operación se pone en el Bus A.

Sumar los registros CX y DX y almacenar el resultado en DX

CX_{out}, G_{on}, Y_{in}

$DX_{out}, ADDsa, ALU_{out}, DX_{in}$

Not BX : BX_{out}, NOT, BX_{in}

Nota: Para que no haya conflicto de información se deben tener en cuenta que :

- No se debe habilitar la compuerta de salida de más de un registro a la vez, ya que el contenido del bus sería impredecible.
- No operar con la ALU (ALU_{out}) y a la vez habilitar la compuerta de comunicación G, debido a que se produciría en el Bus A una situación análoga a la anterior en el Bus B.

4.3 Ciclo de Fetch

El ciclo de Fetch para esta configuración queda de la siguiente forma :

1. $RPI_{out}, G_{on}, RDM_{in}, READ$
 2. $RPI_{out}, CLEAR Y, CARRY_{in=1}, ADDsa, ALU_{out}, RPI_{in}, WAIT_{fmc}$
 3. $RBM_{out}, G_{on}, RI_{Lin}$
 4. $RPI_{out}, G_{on}, RDM_{in}, READ$
 5. $RPI_{out}, CLEAR Y, CARRY_{in=1}, ADDsa, ALU_{out}, RPI_{in}, WAIT_{fmc}$
 6. $RBM_{out}, G_{on}, RI_{Hin}$
- ===== Decodificación =====

Compare con el Fetch correspondiente a la configuración de un bus.

4.4 Instrucciones

Recordemos que la microrutina en la microinstrucción 300 resuelve la dirección de un operando.

ADD CX, BX (Suma de registros)

ADD	CX	BX
-----	----	----

FETCH

===== Decodificación =====

6) $EST_{Hout}, G_{on}, X_{in}$

7) CX_{out}, G_{on}, Y_{in}

8) $BX_{out}, ADDsa, ALU_{out}, CX_{in}$

10) $X_{out}, G_{on}, EST_{in}, END$

JG ROT (Bifurcación por mayor)

JG	ROT
----	-----

FETCH

===== Decodificación =====

6) $EST_{Hout}, G_{on}, X_{in}$

7) IF $\sim P$, BRANCH 90

8) $RPI_{out}, G_{on}, RDM_{in}, READ$

9) $RPI_{out}, CLEAR Y, CARRY_{in=1}, ADDsa, ALU_{out}, RPI_{in}, WAIT_{fmc}$

10) $RBM_{out}, G_{on}, OP1_{Lin}, TRA_{Lin}$

11) $RPI_{out}, G_{on}, RDM_{in}, READ$

12) $RPI_{out}, CLEAR Y, CARRY_{in=1}, ADDsa, ALU_{out}, RPI_{in}, WAIT_{fmc}$

13) $RBM_{out}, G_{on}, OP1_{Hin}, TRA_{Hin}, PC_{aux}=14, BRANCH 300$

14) $TRA_{out}, G_{on}, RPI_{in}, END$

90) $RPI_{out}, CLEAR Y, CARRY_{in=1}, ADDsa, ALU_{out}, RPI_{in}$

91) $RPI_{out}, CLEAR Y, CARRY_{in=1}, ADDsa, ALU_{out}, RPI_{in}, END$

Así como se han analizado dos tipos de configuraciones, en forma análoga pueden estudiarse otros tipos de estructuras.

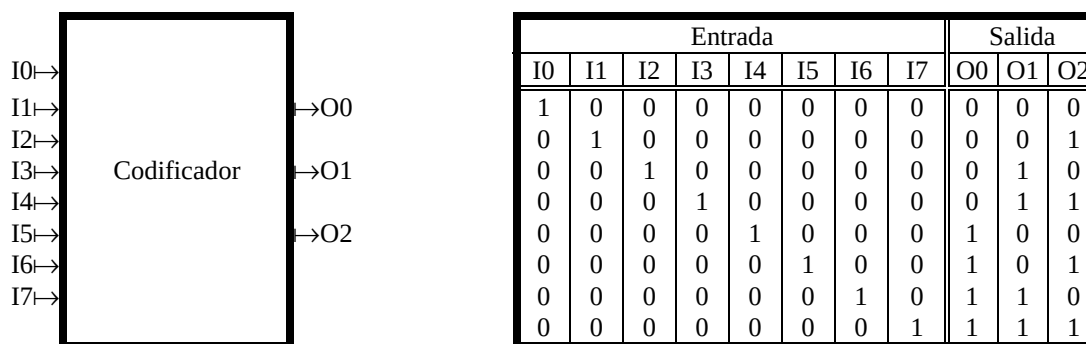
5. Optimización

5.1 Codificadores y decodificadores

En la transmisión de señales se utiliza una línea por cada señal. Así, para transmitir n señales serán necesarias n líneas, lo cual en ciertas ocasiones resulta muy costoso y puede llegar a ser dificultosa su construcción.

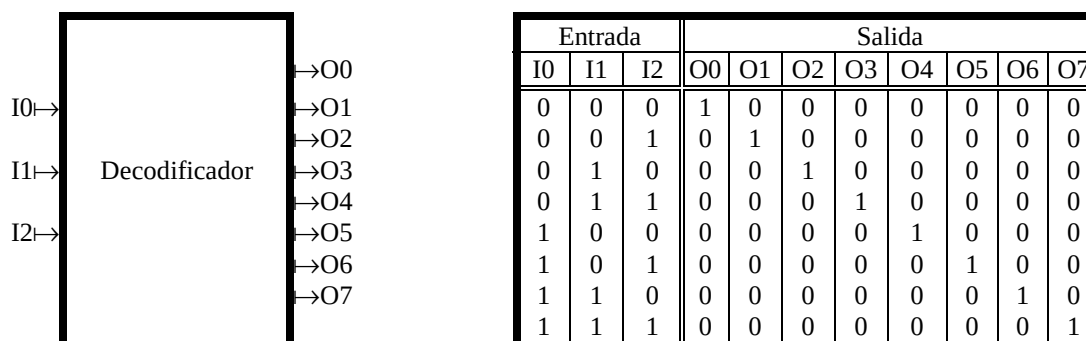
Para solucionar este inconveniente y siempre que las señales no ocurran simultáneamente, se puede utilizar un codificador que reduzca la cantidad de líneas que se transmiten, y luego un decodificador que hace el proceso inverso. El codificador tiene como entrada n líneas y como salida $\log_2 n$.

Por ejemplo un codificador 8 a 3 será :



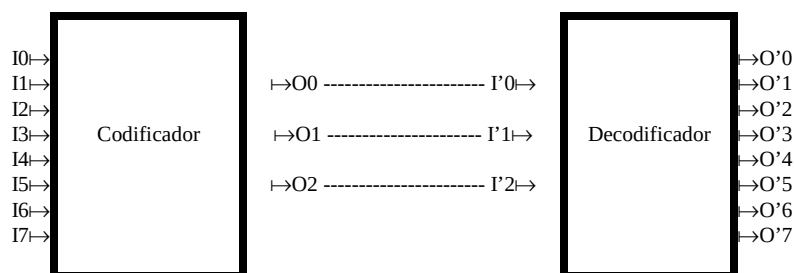
Este codificador posee 8 entradas (de I0 a I7) y 3 salidas (de O0 a O2), y en la tabla se muestra la codificación de la entrada. Notar que no figura en la tabla más de una entrada activada, entonces si llega a ocurrir esto, no podrá ser reconocida y será incierta la salida, por lo cual se concluye que dicha entrada no está permitida.

El caso inverso es un decodificador, el cual tendrá m entradas y 2^m salidas. Entonces el esquema y la tabla de un decodificador 3 a 8 es :



El esquema completo de transmisión puede ser representado como :

Microprogramación



Donde $m = \log_2 n$ o lo que es lo mismo $n = 2^m$.

5.2 Superposición de microoperaciones

Las microinstrucciones están compuestas por microoperaciones, donde cada microoperación representa una señal a un dispositivo como ser la memoria, la ALU o una compuerta. Dichas señales tomarán valores '0' o '1' donde '0' significa 'ausencia de señal' y '1' el envío de la misma. Así se tiene una secuencia de ceros y unos que representan una microinstrucción donde cada 0 o 1 indica la realización o no de una microoperación respectivamente.

Nota: De aquí en adelante se trabajará con el esquema de un bus a menos que se indique lo contrario.

Si se toma el siguiente microprograma, que es el Fetch :

1. RPI_{out} , RDM_{in} , READ, CLEAR Y, $CARRY_{in=1}$, $ADDsa$, Z_{in}
 2. Z_{out} , RPI_{in} , $WAIT_{fmc}$
 3. RBM_{out} , RI_{Lin}
 4. RPI_{out} , RDM_{in} , READ, CLEAR Y, $CARRY_{in=1}$, $ADDsa$, Z_{in}
 5. Z_{out} , RPI_{in} , $WAIT_{fmc}$
 6. RBM_{out} , RI_{Hin}
- ===== Decodificación =====

la configuración de bits será la siguiente :

μoperac. μinstruc.	RPI_{in}	RPI_{out}	Z_{in}	Z_{out}	RBM_{out}	RDM_{in}	RI_{Lin}	RI_{Hin}	$ADDsa$	READ	$WAIT_{fmc}$	CLEAR Y	$CARRY_{in=1}$
0	0	1	1	0	0	1	0	0	1	1	0	1	1
1	1	0	0	1	0	0	0	0	0	0	1	0	0
2	0	0	0	0	1	0	1	0	0	0	0	0	0
3	0	1	1	0	0	1	0	0	1	1	0	1	1
4	1	0	0	1	0	0	0	0	0	0	1	0	0
5	0	0	0	0	1	0	0	1	0	0	0	0	0

Para este microprograma se necesita como mínimo 13 bits, para cada microoperación. Si se considera el conjunto de microoperaciones usadas para todos los microprogramas, se puede

Microprogramación

observar que dicho conjunto es muy grande, lo cual implica una gran cantidad de bits. Se verá a continuación la forma de reducir la cantidad de bits.

Del análisis de la tabla anterior se puede ver que hay microoperaciones que no se ejecutan simultáneamente, como por ejemplo Z_{in} y Z_{out} o RPI_{out} , Z_{out} y RBM_{out} . Estas microoperaciones que no se ejecutan simultáneamente pueden ser agrupadas en un campo codificado. Se pueden agrupar las siguientes microinstrucciones en los siguientes campos:

Campo A : RPI_{out} , Z_{out} , RBM_{out}

Campo B : RPI_{in} , Z_{in} , RI_{Lin} , RI_{Hin}

Campo C : READ, $WAIT_{fmc}$

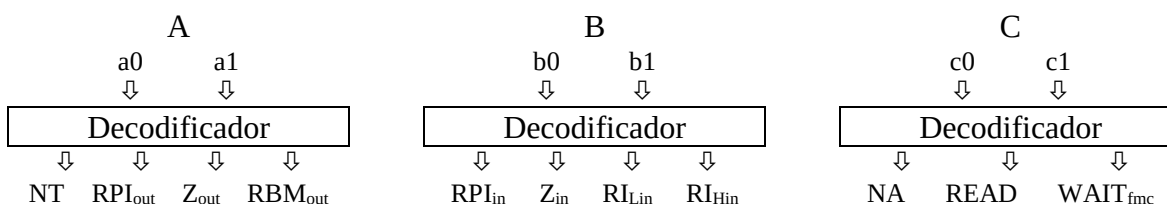
La codificación de los campos utilizados es :

Campo A		Campo B		Campo C	
00	No transfiere	00	RPI_{in}	00	No opera
01	RPI_{out}	01	Z_{in}	01	READ
10	Z_{out}	10	RI_{Lin}	10	$WAIT_{fmc}$
11	RBM_{out}	11	RI_{Hin}	11	Sin uso

Como se observa para estas microoperaciones hubieran sido necesarios 3 bits y de esta forma se utilizan unicamente 2. Con este agrupamiento la tabla queda :

μ operac. μ instruc.	A	B	C	RDM_{in}	ADDsa	CLEAR Y	$CARRY_{in=1}$
0	00	01	01	1	1	1	1
1	10	00	10	0	0	0	0
2	11	10	00	0	0	0	0
3	01	01	01	1	1	1	1
4	10	00	10	0	0	0	0
5	11	11	00	0	0	0	0

La cantidad de bits necesaria para efectuar las microinstrucciones se redujo de 13 a 10. Los bits de los campos A, B y C van a un decodificador de la siguiente forma :



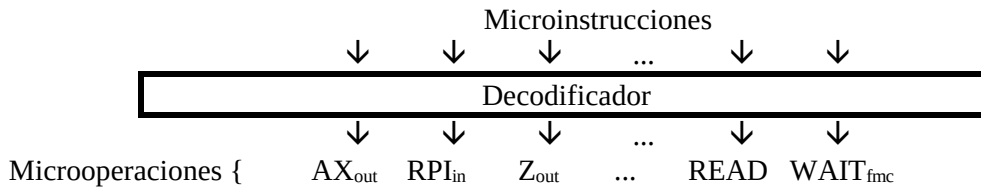
5.3 Microprogramación horizontal, vertical y mixta

En un esquema de organización horizontal es posible efectuar cualquier combinación de microoperaciones simultáneamente, es decir, dentro de una microinstrucción. En este caso las microinstrucciones son más poderosas, lo cual implica mayor rapidez en la ejecución de

Microprogramación

una instrucción, pero afecta al requerimiento de hardware paralelo para el manejo de la ejecución de microinstrucciones (son necesarios una gran cantidad de bits).

En la microprogramación vertical sólo se puede ejecutar una microoperación por microinstrucción, lo cual simplifica el manejo de hardware paralelo, pero perjudica el tiempo de ejecución de cada instrucción. La forma de utilizar este esquema es codificar todas las microoperaciones.



Las organizaciones horizontal y vertical representan dos extremos en control microprogramado. La organización mixta combina las dos anteriores tomando en cuenta el tiempo de ejecución de una instrucción y el manejo de hardware paralelo. La idea es agrupar todas aquellas microoperaciones mutuamente excluyentes en un conjunto que cumpla con las características de la función que realizan.

Entonces, con este criterio, es posible agrupar las operaciones de la ALU en un grupo, y en otros las operaciones con la memoria, las de bifurcación, las condicionales (IF), las compuertas de salida de los registros y en varios grupos las de entrada (según su uso).

Del análisis de una tabla similar a la efectuada anteriormente, que incluya todas las microinstrucciones, se puede determinar y completar el agrupamiento más conveniente de las microoperaciones.

5.4 Aplicaciones de una organización mixta

Se agruparán algunas de las microoperaciones usadas en el presente apunte en funciones. Por ejemplo: señales que llegan a la ALU ocurren una por vez, por lo que se las puede agrupar en F1 de la siguiente manera:

F1 :	00000	—>	No actúa
	00001	—>	ADDs
	00010	—>	ADDc
	00011	—>	ADDsa
	00100	—>	ADDca
	00101	—>	SUBs
	00110	—>	SUBc
	00111	—>	SUBsa
	01000	—>	SUBca
	01001	—>	MULTs
	01010	—>	MULTc
	01011	—>	DIVs
	01100	—>	DIVc
	01101	—>	SHIFT _L
	01110	—>	SHIFT _R
	01111	—>	AND
	10000	—>	OR

Microprogramación

10001	—>	XOR
10010	—>	NOT

Las operaciones de memoria pueden ser agrupadas en F2:

F2:	00	—>	No actúa
	01	—>	READ
	10	—>	WRITE

Las de condición, el WAIT y el CARRY en F3, F4 y F5 respectivamente :

F3:	000	—>	No actúa
	001	—>	Z
	010	—>	N
	011	—>	~N
	100	—>	P
	101	—>	~P
	110	—>	C
	111	—>	~C

F4:	0	—>	No actúa
	1	—>	WAIT _{fmc}

F5:	00	—>	No actúa
	01	—>	CARRY _{in=0}
	10	—>	CARRY _{in=1}

Las de las compuertas de salida se podrán agrupar en F6 debido a que no se habilitará más de una a la vez.

F6 :	0000	—>	No actúa
	0001	—>	AX _{out}
	0010	—>	BX _{out}
	0011	—>	CX _{out}
	0100	—>	DX _{out}
	0101	—>	RBM _{out}
	0110	—>	OP1 _{out}
	0111	—>	OP2 _{out}
	1000	—>	TRA _{out}
	1001	—>	EST _{out}
	1010	—>	AUX1 _{out}
	1011	—>	AUX2 _{out}

Las de las compuertas de entrada no cumplen ninguna regla o restricción en su utilización, entonces no es posible agruparlas según su función. Esto no quita que efectuando un detallado análisis de las microinstrucciones implementadas sea posible encontrar microoperaciones de entrada mutuamente excluyentes.

Con el agrupamiento realizado se logra reducir de 44 bits (uno para cada microoperación) a 17 bits, sin restringir la capacidad de las microinstrucciones.

Microprogramación

Haciendo un balance entre restringir la capacidad de las microinstrucciones y aumentar el tiempo de ejecución de cada microinstrucción o simplificar la construcción del hardware y así reducir su costo se decide cuál es la mejor combinación de diseño.